MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH AND STUDIES

DSA PRACTICAL FILE

NAME	**:**	**MEENAL**
ROLL NO	**:**	**26MCAP101N42**
CLASS	**:**	**MCA1A**
SUBMITTED BY	**:**	**MEENAL**
SUBMITTED TO	**:**	**DR SACHIN SHARMA**

Index

1. Insert a new element in an array
2. Delete an element from an array
3. Find the product of two matrices
4. Sum of diagonal and non-diagonal elements in a matrix
5. Marks of 5 students in 3 subjects and their percentage
6.1 Stack operations
6.2 Reverse of a string using stack
6.3 Parenthesis matching using stack
7.1 General queue operations
7.2 Circular queue operations

1. Insert a new element in an array

Program

```
# Program 1: Insert a new element in an array

n = int(input("How many elements are in the array? "))
arr = []
for i in range(n):
    x = int(input("Enter element " + str(i + 1) + ": "))
    arr.append(x)

print("Array before insertion:", arr)

pos = int(input("Enter the position where you want to insert (starting from 1): "))
value = int(input("Enter the new element: "))

if pos < 1 or pos > n + 1:
    print("Invalid position!")
else:
    # make space at the end
    arr.append(0)
    # shift elements to the right, starting from the last one
    i = n
    while i >= pos:
        arr[i] = arr[i - 1]
        i = i - 1
    # put the new element in its place
    arr[pos - 1] = value
    print("Array after insertion:", arr)
```

Output

```
How many elements are in the array? 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Array before insertion: [10, 20, 30, 40, 50]
Enter the position where you want to insert (starting from 1): 3
Enter the new element: 25
Array after insertion: [10, 20, 25, 30, 40, 50]
```

2. Delete an element from an array

Program

```
# Program 2: Delete an element from an array

n = int(input("How many elements are in the array? "))
arr = []
for i in range(n):
    x = int(input("Enter element " + str(i + 1) + ": "))
    arr.append(x)

print("Array before deletion:", arr)

pos = int(input("Enter the position of the element to delete (starting from 1): "))

if pos < 1 or pos > n:
    print("Invalid position!")
else:
    print("Deleting element:", arr[pos - 1])
    # shift all elements after the position one step to the left
    for i in range(pos - 1, n - 1):
        arr[i] = arr[i + 1]
    # remove the extra last element
    arr.pop()
    print("Array after deletion:", arr)
```

Output

```
How many elements are in the array? 5
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Enter element 5: 50
Array before deletion: [10, 20, 30, 40, 50]
Enter the position of the element to delete (starting from 1): 2
Deleting element: 20
Array after deletion: [10, 30, 40, 50]
```

3. Find the product of two matrices

Program

```
# Program 3: Find the product of two matrices

r1 = int(input("Enter rows of first matrix: "))
c1 = int(input("Enter columns of first matrix: "))
r2 = int(input("Enter rows of second matrix: "))
c2 = int(input("Enter columns of second matrix: "))

# Matrices can be multiplied only if columns of A = rows of B
if c1 != r2:
    print("Multiplication is not possible!")
else:
    A = []
    B = []

    print("Enter elements of first matrix:")
    for i in range(r1):
        row = []
        for j in range(c1):
            row.append(int(input("A[" + str(i) + "][" + str(j) + "]: ")))
        A.append(row)

    print("Enter elements of second matrix:")
    for i in range(r2):
        row = []
        for j in range(c2):
            row.append(int(input("B[" + str(i) + "][" + str(j) + "]: ")))
        B.append(row)

    # result matrix filled with zeros
    result = []
    for i in range(r1):
        result.append([0] * c2)

    # multiply
    for i in range(r1):
        for j in range(c2):
            for k in range(c1):
                result[i][j] = result[i][j] + A[i][k] * B[k][j]

    print("Product of the two matrices:")
    for row in result:
        for value in row:
            print(value, end="\t")
        print()
```

Output

```
Enter rows of first matrix: 2
Enter columns of first matrix: 3
Enter rows of second matrix: 3
Enter columns of second matrix: 2
Enter elements of first matrix:
A[0][0]: 1
A[0][1]: 2
A[0][2]: 3
A[1][0]: 4
A[1][1]: 5
A[1][2]: 6
Enter elements of second matrix:
B[0][0]: 7
B[0][1]: 8
B[1][0]: 9
B[1][1]: 10
B[2][0]: 11
B[2][1]: 12
Product of the two matrices:
58      64
139     154
```

4. Sum of diagonal and non-diagonal elements in a matrix

Program

```
# Program 4: Sum of diagonal and non-diagonal elements of a matrix

n = int(input("Enter the size of the square matrix (n x n): "))

matrix = []
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input("Enter element [" + str(i) + "][" + str(j) + "]: ")))
    matrix.append(row)

print("The matrix is:")
for row in matrix:
    for value in row:
        print(value, end="\t")
    print()

diagonal_sum = 0
non_diagonal_sum = 0

for i in range(n):
    for j in range(n):
        if i == j:
            diagonal_sum = diagonal_sum + matrix[i][j]
        else:
            non_diagonal_sum = non_diagonal_sum + matrix[i][j]

print("Sum of diagonal elements =", diagonal_sum)
print("Sum of non-diagonal elements =", non_diagonal_sum)
```

Output

```
Enter the size of the square matrix (n x n): 3
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9
The matrix is:
1       2       3
4       5       6
7       8       9
Sum of diagonal elements = 15
Sum of non-diagonal elements = 30
```

5. Marks of 5 students in 3 subjects and their percentage

Program

```
# Program 5: Enter marks of 5 students in 3 subjects and find their percentage

students = 5
subjects = 3
max_marks = 100   # each subject is out of 100

for s in range(students):
    print("\nStudent", s + 1)
    total = 0
    for sub in range(subjects):
        m = float(input("Enter marks of subject " + str(sub + 1) + ": "))
        total = total + m

    percentage = (total / (subjects * max_marks)) * 100
    print("Total marks =", total, "out of", subjects * max_marks)
    print("Percentage =", round(percentage, 2), "%")
```

Output

```
Student 1
Enter marks of subject 1: 78
Enter marks of subject 2: 85
Enter marks of subject 3: 90
Total marks = 253.0 out of 300
Percentage = 84.33 %

Student 2
Enter marks of subject 1: 65
Enter marks of subject 2: 70
Enter marks of subject 3: 72
Total marks = 207.0 out of 300
Percentage = 69.0 %

Student 3
Enter marks of subject 1: 92
Enter marks of subject 2: 88
Enter marks of subject 3: 95
Total marks = 275.0 out of 300
Percentage = 91.67 %

Student 4
Enter marks of subject 1: 55
Enter marks of subject 2: 60
Enter marks of subject 3: 58
Total marks = 173.0 out of 300
Percentage = 57.67 %

Student 5
Enter marks of subject 1: 80
Enter marks of subject 2: 75
Enter marks of subject 3: 85
Total marks = 240.0 out of 300
Percentage = 80.0 %
```

6.1 Stack operations

Program

```
# Program 6.1: Stack operations (push, pop, peek, display)

stack = []
size = int(input("Enter the size of the stack: "))

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek (top element)")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(stack) == size:
            print("Stack Overflow! Stack is full.")
        else:
            x = input("Enter the element to push: ")
            stack.append(x)
            print(x, "pushed into the stack")

    elif choice == 2:
        if len(stack) == 0:
            print("Stack Underflow! Stack is empty.")
        else:
            x = stack.pop()
            print(x, "popped from the stack")

    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element is:", stack[-1])

    elif choice == 4:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements (top to bottom):")
            for i in range(len(stack) - 1, -1, -1):
                print(stack[i])

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Wrong choice, try again.")
```

Output

```
Enter the size of the stack: 3

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 10
10 pushed into the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 20
20 pushed into the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
```

```
5. Exit
Enter your choice: 1
Enter the element to push: 30
30 pushed into the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 1
Stack Overflow! Stack is full.

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 40
Wrong choice, try again.

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
30
20
10

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 3
Top element is: 30

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 2
30 popped from the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 2
20 popped from the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 2
10 popped from the stack

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
Enter your choice: 2
Stack Underflow! Stack is empty.

1. Push
2. Pop
3. Peek (top element)
4. Display
5. Exit
```

```
Enter your choice: 5
Exiting...
```

6.2 Reverse of a string using stack

Program

```
# Program 6.2: Reverse a string using a stack

text = input("Enter a string: ")

stack = []

# push every character into the stack
for ch in text:
    stack.append(ch)

# pop characters one by one (last character comes out first)
reversed_text = ""
while len(stack) > 0:
    reversed_text = reversed_text + stack.pop()

print("Original string:", text)
print("Reversed string:", reversed_text)
```

Output

```
Enter a string: Manav Rachna
Original string: Manav Rachna
Reversed string: anhcaR vanaM
```

6.3 Parenthesis matching using stack

Program

```
# Program 6.3: Parenthesis matching using a stack

expression = input("Enter an expression: ")

stack = []
balanced = True

for ch in expression:
    # opening bracket -> push
    if ch == "(" or ch == "{" or ch == "[":
        stack.append(ch)

    # closing bracket -> check with the top of the stack
    elif ch == ")" or ch == "}" or ch == "]":
        if len(stack) == 0:
            balanced = False
            break
        top = stack.pop()
        if (ch == ")" and top != "(") or \
           (ch == "}" and top != "{") or \
           (ch == "]" and top != "["):
            balanced = False
            break

# if something is still left in the stack, brackets are not closed
if len(stack) != 0:
    balanced = False

if balanced:
    print("Parentheses are matched (balanced).")
else:
    print("Parentheses are NOT matched.")
```

Output 1

```
Enter an expression: {a + (b * c)} - [d / e]
Parentheses are matched (balanced).
```

Output 2

```
Enter an expression: ((a + b) * c
Parentheses are NOT matched.
```

7.1 General queue operations

Program

```
# Program 7.1: General (simple) Queue operations

queue = []
size = int(input("Enter the size of the queue: "))

while True:
    print("\n1. Enqueue (insert)")
    print("2. Dequeue (delete)")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(queue) == size:
            print("Queue is full!")
        else:
            x = input("Enter the element to insert: ")
            queue.append(x)          # insert at the rear
            print(x, "inserted into the queue")

    elif choice == 2:
        if len(queue) == 0:
            print("Queue is empty!")
        else:
            x = queue.pop(0)         # delete from the front
            print(x, "deleted from the queue")

    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty!")
        else:
            print("Queue elements (front to rear):", queue)

    elif choice == 4:
        print("Exiting...")
        break

    else:
        print("Wrong choice, try again.")
```

Output

```
Enter the size of the queue: 3

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 5
5 inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 10
10 inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: 15
15 inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Queue is full!
1. Enqueue (insert)
```

```
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 20
Wrong choice, try again.

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 3
Queue elements (front to rear): ['5', '10', '15']

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
5 deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
10 deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 3
Queue elements (front to rear): ['15']

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
15 deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
Queue is empty!

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 4
Exiting...
```

7.2 Circular queue operations

Program

```
# Program 7.2: Circular Queue operations

size = int(input("Enter the size of the circular queue: "))
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n1. Enqueue (insert)")
    print("2. Dequeue (delete)")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        # queue is full when rear is just behind front
        if (rear + 1) % size == front:
            print("Queue is full!")
        else:
            x = input("Enter the element to insert: ")
            if front == -1:          # first element
                front = 0
            rear = (rear + 1) % size
            queue[rear] = x
            print(x, "inserted into the queue")

    elif choice == 2:
        if front == -1:
            print("Queue is empty!")
        else:
            print(queue[front], "deleted from the queue")
            queue[front] = None
            if front == rear:        # last element was removed
                front = -1
                rear = -1
            else:
                front = (front + 1) % size

    elif choice == 3:
        if front == -1:
            print("Queue is empty!")
        else:
            print("Queue elements (front to rear):", end=" ")
            i = front
            while True:
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % size
            print()

    elif choice == 4:
        print("Exiting...")
        break

    else:
        print("Wrong choice, try again.")
```

Output

```
Enter the size of the circular queue: 3

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: A
A inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
```

```
Enter the element to insert: B
B inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: C
C inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Queue is full!

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 3
Queue elements (front to rear): A B C

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
A deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 1
Enter the element to insert: D
D inserted into the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 3
Queue elements (front to rear): B C D

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
B deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
C deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
D deleted from the queue

1. Enqueue (insert)
2. Dequeue (delete)
3. Display
4. Exit
Enter your choice: 2
Queue is empty!

1. Enqueue (insert)
2. Dequeue (delete)
```

```
3. Display
4. Exit
Enter your choice: 4
Exiting...
```